Breaking the Observation Ceiling: Conformal Fusion of Trace Signals for Detecting Prompt-Injection Compromise in LLM Agents

Quang-Vinh Dang[a]

[a]*British University Vietnam, Hung Yen, Vietnam*

Abstract

Large language model (LLM) agents that call external tools ingest untrusted content and fold it into the context that drives their next action, exposing them to indirect prompt injection. Runtime monitors that watch the agent's black-box execution trace are attractive because they are model-agnostic and cheap, but the literature has converged on a single family of evidence: detecting whether untrusted text *looks like an instruction*. We show that this choice imposes a hard information limit and that crossing it requires a different kind of evidence.

We make four contributions. First, we introduce **SENTRY-Fuse**, a monitor that converts heterogeneous trace signals—two that score the *observation* stream and two that score the *action* stream—into conformal tail surprisals against a benign calibration split and fuses them additively. The construction is scale-free, strictly monotone in every component, and requires no tuned weights; we prove that these properties are exactly what a fusion rule needs, and we exhibit a widely-used alternative normalisation that provably destroys signal. Second, we show that action-side evidence is *complementary* rather than redundant: on the pooled corpus SENTRY-Fuse attains a true-positive rate of 0.902 at a 3.66% false-positive rate, strictly above the 0.867 attainable from payload visibility alone, and on a matched GPT-4o-mini corpus it reaches 0.960 ± 0.010, exceeding the strongest published trace-only figure (0.9575). Third, we characterise the **detectability ceiling** of trace-only monitoring: over 90 genuinely compromised trajectories, 7.8% leave no trace evidence whatsoever, bounding every such monitor at 0.922—a bound that

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

published numbers above 0.92 on this benchmark exceed and must therefore explain. Fourth, we document a label-polarity pitfall in the standard AgentDojo harness that inverts the compromised and resisted classes, which silently inverted our own results, and we give the ground-truth procedure that settles it. We report our negative results in full, including three rejected signals and the finding that a hand-tuned threshold dominates the anytime-valid e-detector we began from. All code, trajectories and scripts are released.

Keywords: LLM agents, indirect prompt injection, runtime monitoring, conformal prediction, anomaly detection, AI agent security

---

1. Introduction

Autonomous agents built on large language models now read email, browse the web, query databases and move money on a user's behalf. To act, they consume *untrusted* content—web pages, documents, tool outputs—and fold it directly into the context that determines their next tool call. This is the attack surface exploited by *indirect prompt injection*: an adversary plants an instruction inside data the agent will later read, and the agent, unable to separate trusted instructions from untrusted data, follows it (Zhan et al., 2024; Debenedetti et al., 2024). As agents move from demos into production, a runtime monitor—something that watches the live execution trace and halts or escalates when behaviour departs from what is safe—becomes as important as the agent itself.

Monitors that observe only the black-box trace are especially attractive in practice. They need no access to model weights or activations, they impose no retraining cost, they compose with whichever model a deployment happens to use, and they can be dropped in front of a third-party agent. A substantial line of work therefore builds detectors over traces (Wang et al., 2025; Liu et al., 2025; Jia et al., 2025). Almost all of it, however, rests on one kind of evidence: examining the untrusted content the agent read and asking whether it *looks like an instruction*. Keyword and perplexity heuristics do this lexically; fine-tuned detectors and LLM judges do it semantically.

The observation that started this work. We began from that same premise and built a monitor on it. It performed well on the standard benchmark attack and then collapsed, to chance, when we changed the attack template. Our first explanation was that the new attack was phrased too subtly. That

explanation was *wrong*: the new payloads were, if anything, more imperative, and they appeared in the trace more often. The collapse came from our own normalisation step. Diagnosing that failure honestly led us to a more general question, which organises this paper:

> *What, exactly, can a monitor learn from a black-box agent trace, and where does that information run out?*

Answering it requires separating two objectives that the literature routinely conflates. *Attempt detection* asks whether untrusted instructions entered the agent's context at all; *compromise detection* asks whether the agent actually did the attacker's bidding. They have different positive classes, different ceilings, and—as we show—different optimal evidence. A monitor tuned for the first is not automatically good at the second.

Contributions. Our contributions are as follows.

1. **SENTRY-Fuse, a conformal fusion monitor** (Section 3.4). We combine four trace signals—two scoring the observation stream and two scoring the action stream—by mapping each to its conformal tail surprisal against a benign calibration split and summing. We show (Propositions 2–3) that the construction is distribution-free valid, scale-free and strictly monotone in every component, and that the excess-over-maximum normalisation used in an earlier version of this work is many-to-one and can annihilate an entire attack family—which it did, measurably (Section 3.6).
2. **Action-side evidence is complementary, not redundant** (Section 5). Signals that audit *what the agent did* carry information that signals scoring *what the agent read* cannot. The fused monitor exceeds the payload-visibility ceiling, and on a matched corpus it exceeds the strongest published trace-only number.
3. **The detectability ceiling of trace-only monitoring** (Section 5.4). We formalise and measure the limit: some successful injections leave no trace evidence at all, capping any trace-only detector. Theorem 5 makes the bound precise; empirically it is 0.922, and we reach 97.8% of it.
4. **Methodological findings and honest negatives** (Sections 4.4 and 5.5). We document an label-polarity trap in the standard evaluation harness

that inverts the compromised and resisted classes; three signals we implemented and rejected; and the finding that a hand-picked threshold dominates the anytime-valid e-detector we started from, which we report rather than omit.

Every number and figure reported here is reproducible from the released corpora without any model API access, since all trajectories and cached judge scores are published alongside the code.

2. Background and Related Work

2.1. Indirect prompt injection

Direct prompt injection manipulates the user-supplied prompt; *indirect* prompt injection is the more dangerous variant for agents, because the adversary never talks to the agent at all. Greshake et al. (2023) introduced the threat and gave the first systematic treatment of it. Their central observation is the one this entire literature rests on: once an LLM is augmented with retrieval, *processing* untrusted retrieved data becomes analogous to *executing* arbitrary code, because the boundary between data and instructions is not represented anywhere in the model's input. They derive a taxonomy of resulting threats—information gathering, fraud, intrusion, malware (including prompts that propagate as worms), manipulated content, and availability attacks—and a taxonomy of delivery methods, distinguishing *passive* injection (planting content that retrieval will find), *active* injection (sending content, e.g. by email, that the agent will process), *user-driven* injection (tricking the user into pasting it), and *hidden* injection (multi-stage or encoded payloads). They demonstrate feasibility against real deployed systems including Bing Chat and GitHub Copilot. Our threat model (Section 3) is the passive/active case, and the compromises we detect fall predominantly into their information-gathering and fraud categories.

The consequences scale with the agent's privileges: an agent that can send email can exfiltrate; an agent that can move money can steal. Zhan et al. (2024) systematise this for tool-integrated agents specifically.

Wallace et al. (2024) argue that the root cause is the absence of a privilege ordering over instruction sources, and train models to respect an explicit *instruction hierarchy*. This mitigates but does not eliminate the problem, since the model must still infer which span of a long context is privileged. Nasr et al. (2025) show that defences evaluated only against static attacks

tend to collapse under adaptive attacks that optimise against the defence, and argue that adaptive evaluation should be the norm—a caution we take seriously in Section 6.1.

2.2. Benchmarks for agent security

Progress here is benchmark-driven, and the choice of benchmark determines what "detection" even means.

AgentDojo (Debenedetti et al., 2024) is the most widely used harness for our setting. It provides four tool environments (banking, workspace, travel, Slack), a set of *user tasks* defining legitimate goals, and a set of *injection tasks* defining attacker goals, together with attack templates that place the attacker's directive into environment content. Crucially, it scores two things separately: *utility*, whether the user's task was accomplished, and *security*, whether the *injection* task's goal was accomplished. Section 4.4 shows that the direction of this second flag is easy to invert, with severe consequences.

InjecAgent (Zhan et al., 2024) contributes a distinct family of injection templates aimed at tool-integrated agents, which we use to test whether a detector generalises beyond the template it was developed on. **τ-bench** (Yao et al., 2025) supplies realistic benign multi-turn tool-use trajectories in customer-service domains; it has no attack suite, so we use it purely to enlarge and diversify the benign reference. **WebArena** (Zhou et al., 2024) provides realistic web environments and is complementary but not injection-focused.

AgentHarm (Andriushchenko et al., 2025) deserves separate mention because it is easy to mistake for our setting. It comprises 110 base (and 440 augmented) explicitly malicious agent behaviours across 11 harm categories over 104 tools, and it scores whether an agent both refuses *and* retains capability. Its threat model, however, is agent *misuse*: the harmful request comes from the user directly. Ours is indirect injection, where the user is benign and the harm arrives through third-party content—a distinction the AgentHarm authors themselves draw when contrasting their benchmark with AgentDojo. The two are complementary and a complete guardrail needs both; we evaluate only the injection case, which Section 6.1 records as a scope limitation.

Li et al. (2026) survey the space and note substantial inconsistency in how safety benchmarks define and measure success—a concern our Sections 4.4 and 5.4 substantiate concretely.

2.3. Defences: prevention

Prevention redesigns the agent, or retrains the model, so that injected instructions cannot influence control flow. It is worth separating three mechanisms, because each fails differently and each leaves a different residual role for monitoring.

Marking provenance in the prompt. **Spotlighting** (Hines et al., 2024) observes that an LLM receives multiple input sources concatenated into one token stream with no signal of which span came from where, and supplies that signal by transforming untrusted spans: delimiting them with explicit markers, *datamarking* them by interleaving a reserved token throughout the untrusted text, or encoding them (e.g. base64) so that the model can still read them but their instruction-like surface form is disrupted. On GPT-family models this reduces attack success from above 50% to below 2% with little utility loss. The approach is attractively cheap—it is prompt engineering, requiring no training—but it relies on the model honouring the marking, which is a behavioural rather than an architectural guarantee.

Changing the interface. **StruQ** (Chen et al., 2025a) makes the sharpest conceptual argument in this literature. Prompt injection, they observe, is the latest instance of a recurring vulnerability class in which control and data share one channel: exactly as SQL injection arises because a query string mixes control with data, and cross-site scripting because an HTML page does, prompt injection arises because the LLM API is *unsafe by design*, expecting developers to concatenate the prompt and the data. Their remedy is the analogue of prepared statements: a *structured query* presenting instruction and data as two separate inputs, implemented by a filtering front-end using reserved delimiter tokens together with *structured instruction tuning*, which fine-tunes the model on examples where instructions appear in the data portion and must be ignored. This drives manually-crafted attacks below 2% and substantially weakens optimisation-based ones (TAP 97% $\rightarrow$ 9%).

Aligning against the injection. **SecAlign** (Chen et al., 2025b) identifies the weakness of the fine-tuning approach: training only to *prefer* the correct response underspecifies what an incorrect response looks like, so security does not generalise to unseen attacks—StruQ, they report, still exceeds 50% attack success under an attack that optimises the injection. SecAlign therefore builds a preference dataset of (injected input, secure response, insecure

response) triples and applies preference optimisation, so the model is explicitly steered *away* from the injected response. This yields near-0% success against optimisation-free attacks and below 10% against optimisation-based ones, improving on StruQ by more than a factor of four.

Sanitising the content. **CommandSans** (Das et al., 2025) strips imperative content from tool outputs before the agent reads them, a lighter-weight route that requires no retraining.

Enforcing provenance at the system level. **CaMeL** (Debenedetti et al., 2025) extracts the control flow from the trusted user query, tracks data provenance through a capability system, and refuses tool calls that carry untrusted taint into sensitive argument positions. This is the strongest guarantee of the group and the most invasive, since it requires rebuilding the agent around the defence.

Why monitoring remains necessary. Every mechanism above modifies the agent or the model. That is often impossible: deployments frequently wrap a third-party model behind an API, cannot retrain it, and cannot re-architect a vendor's agent loop. Notably, Chen et al. (2025b) explicitly scope their contribution to prevention and set detection-based defences aside; the two lines of work are therefore complementary rather than competing. Prevention narrows the attack surface; monitoring supplies an independent tripwire where prevention is absent, partial, or itself circumvented—and, unlike prevention, it produces an auditable record of what happened, which is what incident response needs.

2.4. Defences: detection

Detection leaves the agent untouched and raises an alarm from a score. **DataSentinel** (Liu et al., 2025) fine-tunes an LLM to detect contaminated inputs, formulated as a minimax game so that the detector is trained against adaptive evasion. **Task Shield** (Jia et al., 2025) checks each action for alignment with the user's stated task—conceptually the closest prior work to our action-side signal, though it is used as a standalone gate rather than as one calibrated component of a fused statistic. **MELON** (Zhu et al., 2025) detects injection by re-executing the trajectory with a masked user prompt and comparing the resulting tool calls, on the principle that a hijacked action recurs even when the real task is removed; this is powerful but pays a full agent re-run per check.

AgentArmor (Wang et al., 2025) is the strongest published *trace-only* detector and our reference point. It lifts the agent trace into a program representation—control-flow, data-flow and program-dependence graphs—and enforces typed security policies over it, reporting 95.75% true positives at a 3.66% false-positive rate. Because it reasons about which untrusted value flowed into which argument of which tool, it is structurally different from a scalar text score, and Section 5.4 argues that its reported operating point exceeds what the trace alone can support on the corpus we study, which is informative about how such numbers should be interpreted.

Recent work also monitors adjacent failure modes: Belenky et al. (2026) detect reward hacking cheaply, Thaman (2026) benchmark tool-use exploits, and Opoku and Banahene (2026) apply conformal risk control to retrieval and tool-use drift.

2.5. Distribution-free calibration and anytime-valid inference

Our fusion rule is built from conformal p-values. Conformal prediction (Vovk et al., 2022) converts any real-valued score into a distribution-free valid p-value using only exchangeability of a calibration sample, which is what lets us combine signals living on incompatible scales without distributional assumptions.

The project began in the adjacent literature on *anytime-valid* inference. E-values and e-processes (Vovk and Wang, 2021; Ramdas et al., 2023; Grünwald et al., 2024) support continuous monitoring: a non-negative process whose expectation is bounded under the null can be inspected at every step, and thresholding it at $1/\alpha$ controls error at level α at every stopping time by Ville's inequality. Shin et al. (2023) extend this to sequential *change* detection with e-detectors, whose average-run-length (ARL) guarantee holds with no independence assumption—attractive here, because an agent's actions are profoundly dependent. Sadhuka et al. (2025) note that when the reference is *learned* the process is no longer an exact e-process and supply a PAC-style thresholding correction, and Waudby-Smith and Ramdas (2023); Page (1954); Lorden (1971) provide the betting and classical change-detection background. Several 2026 works apply anytime-valid ideas to LLM systems (Xie, 2026; Li, 2026; Gauthier et al., 2026) and conformal ideas to agent error attribution (Feng et al., 2026).

We retain conformal calibration, which does real work in our method, but Section 5.5.2 reports that the sequential e-detector machinery did *not* outperform a hand-picked threshold at the data scales we could reach. We therefore

present it as one calibration option rather than as the paper's contribution—a demotion the evidence forced.

3. Threat Model and Method

3.1. Agent execution traces

An agent pursues a user request u by interleaving tool calls and tool results. We write a trace of length T as

$$\mathcal{T} = \big((o_0, a_1), (o_1, a_2), \ldots, (o_{T-1}, a_T)\big), \tag{1}$$

where a_t is the t-th action (a tool name together with its arguments) and o_{t-1} is the tool output the agent observed immediately before choosing a_t. A monitor sees u and $\mathcal{T}$ and nothing else: no model weights, no activations, no logits. We deliberately exclude the agent's own natural-language messages, both because they are not always available in production logging and to keep the setting comparable to prior trace-only work; Section 6.1 shows this exclusion is exactly what the residual undetectable cases exploit.

3.2. Adversary

The adversary controls the textual content of one or more environment resources that the agent may retrieve, and nothing else. They cannot modify u, the tool implementations, the model, or the monitor. Their goal is to cause the agent to execute an *injection task*: an action sequence serving the adversary rather than the user. This is the standard indirect-prompt-injection model (Debenedetti et al., 2024).

3.3. Two distinct detection objectives

Definition 1 (Attempt detection)**.** A trace is *attempt-positive* if injected adversarial content entered the agent's context, whether or not the agent complied.

Definition 2 (Compromise detection)**.** A trace is *compromise-positive* if the agent actually accomplished the adversary's injection task.

Both are legitimate targets. Attempt detection suits a guardrail that should escalate whenever untrusted instructions reach the context, since an agent that resisted today may not tomorrow. Compromise detection suits incident response and is the harder, more consequential question. The two

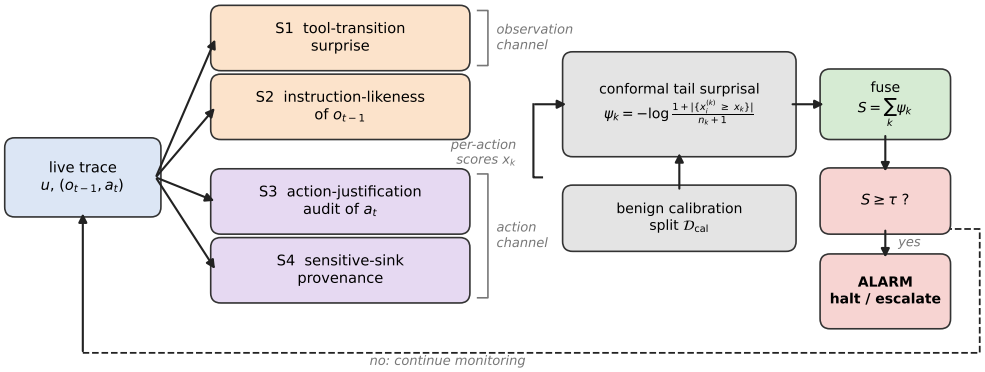

Figure 1: Overview of SENTRY-Fuse. Four signals are computed per action from two distinct evidence channels: the *observation* channel (S1, S2) asks whether untrusted content is behaving like an instruction, while the *action* channel (S3, S4) asks whether the agent's behaviour serves the user's request. Each raw score is mapped to a conformal tail surprisal against a benign calibration split, which makes the heterogeneous signals commensurable and—unlike the excess-over-maximum alternative of Proposition 1—cannot annihilate a component. The surprisals are summed and thresholded; on an alarm the largest term, $\arg\max_k \psi_k$, names the signal that fired, which gives the operator an actionable reason rather than a bare alert.

differ sharply in practice: in our corpus a capable agent resists most attempts, so the attempt-positive class is an order of magnitude larger than the compromise-positive class. Results for one must never be reported as results for the other—a confusion we made ourselves, as Section 4.4 describes.

3.4. Overview of SENTRY-Fuse

Figure 1 summarises the monitor. Each action of the live trace is scored by four signals drawn from two distinct evidence channels; each raw score is converted to a conformal tail surprisal against a benign calibration split; the surprisals are summed and compared against a threshold; and when the monitor fires, the dominant term names the reason. The subsections below define each stage.

3.5. Signals

We compute four per-action signals. Signals S1 and S2 score the *observation* stream, asking whether untrusted content is behaving like an instruction; S3 and S4 score the *action* stream, asking whether the agent's behaviour serves the user.

S1: tool-transition surprise. A Laplace-smoothed bigram model over consecutive tool names, fitted on benign traces, gives

$$s_t^{\mathrm{tr}} = -\log \hat{P}\big(\mathrm{tool}(a_t) \mid \mathrm{tool}(a_{t-1})\big). \tag{2}$$

S2: observation instruction-likeness. Let $\mathrm{IL}(o)$ score how strongly a tool output reads as a directive addressed to the agent. We evaluate two instantiations. The *lexicon* variant is a weighted per-word density of imperative verbs, second-person pronouns, pseudo-markup and exclamation marks. The *judge* variant queries a small LLM with the observation alone and takes a normalised integer rating. Section 5.3 compares them; the judge generalises better across attack templates, at the price of one cached model call per distinct observation.

S3: action-justification audit. Given only the user request u, the tool name and its arguments—and *never* the observations—an LLM judge rates how far the call fails to serve u:

$$s_t^{\mathrm{aj}} = \mathrm{Unjustified}(u,\, a_t) \in [0, 1]. \tag{3}$$

Withholding the observations is a deliberate design choice: it prevents the audit from simply re-detecting the injected text, so S3 is not a restatement of S2 and the two may be fused without circularity.

S4: sensitive-sink provenance. A conjunctive, per-value rule that fires when the agent performs an externally-effectful call whose critical argument traces to untrusted prose:

$$s_t^{\mathrm{sp}} = \mathbf{1}\big[\,\mathrm{Sink}(a_t) \wedge \exists v \in \mathrm{Ids}(a_t) :\ v \notin u \ \wedge\ \mathrm{FirstSeenInProse}(v)\,\big]. \tag{4}$$

Here Sink marks tools with external effect (moving money, sending messages, publishing, editing standing instructions) as opposed to read-only lookups; Ids extracts identifier-like values (IBANs, e-mail addresses, URLs, long digit strings); and FirstSeenInProse holds when the value's first appearance in the trace is inside a long natural-language line rather than a short structured record field. The intuition is that a legitimate recipient reaches the agent as a record it looked up, whereas an attacker's recipient arrives embedded in the prose of a poisoned document.

3.6. Why naive normalisation fails

The four signals live on incompatible scales: s^{tr} is in nats and unbounded, s^{aj} lies in $[0, 1]$, s^{sp} is binary. Summing them raw lets the widest-ranged term dominate. A natural fix, and the one an earlier version of this work used, is to subtract the largest value observed in benign training data,

$$\phi_{\max}(x) = \max\{0,\ x - \beta\}, \qquad \beta = \max_{x' \in \mathcal{D}_{\text{train}}} x', \tag{5}$$

so that anything within the benign range scores zero. This is a trap.

Throughout, AUROC denotes the area under the receiver operating characteristic curve: the probability that a randomly drawn positive scores above a randomly drawn negative, counting ties as one half.

Proposition 1 (Excess-over-maximum can annihilate a signal)**.** *Let $\phi_{\max}$ be as in* (5) *and let $\mathcal{B}, \mathcal{C}$ be the benign and attack score distributions. If $\mathbb{P}_{X \sim \mathcal{C}}(X \le \beta) = 1$, then $\phi_{\max}(X) = 0$ almost surely for both classes and $\text{AUROC}(\phi_{\max}(\mathcal{B}), \phi_{\max}(\mathcal{C})) = 1/2$, irrespective of how well x itself separates the classes.*

Proof. $\phi_{\max}$ maps $(-\infty, \beta]$ to the single point 0. If the attack distribution is supported on $(-\infty, \beta]$ then the transformed attack scores are identically 0; the transformed benign scores are non-negative and equal 0 except on the benign event $\{x > \beta\}$, which has probability zero by the definition of β as the training maximum in the population limit. All comparisons are therefore ties, and AUROC, which scores ties at $1/2$, equals $1/2$. □ □

The premise is not hypothetical. In our data the benign median instruction-likeness is 0.010 but its maximum is 0.613—an outlier—whereas the median for the InjecAgent family is 0.115. Every InjecAgent score therefore fell below β, and a signal that separates that family at AUROC = 0.84 in raw form was reduced to exactly chance. We had published the resulting collapse and attributed it to the attack being subtly phrased. It was our normalisation.

The lesson generalises: a fusion rule must never apply a many-to-one transform to a component signal.

3.7. Conformal surprisal fusion

Definition 3 (Conformal tail surprisal)**.** Let $x_{1:n}$ be benign calibration values for a signal and x a test value. The conformal p-value and surprisal are

$$p(x) = \frac{1 + |\{i : x_i \ge x\}|}{n + 1}, \qquad \psi(x) = -\log p(x). \tag{6}$$

Proposition 2 (Distribution-free validity)**.** *If $x, x_1, \ldots, x_n$ are exchangeable, then $\mathbb{P}(p(x) \leq \alpha) \leq \alpha$ for every $\alpha \in (0,1)$; equivalently $\mathbb{P}(\psi(x) \geq \log(1/\alpha)) \leq \alpha$.*

Proof. By exchangeability the rank of x among $\{x, x_1, \ldots, x_n\}$ is uniform on $\{1, \ldots, n+1\}$. The event $\{p(x) \leq \alpha\}$ is the event that this rank places x among the top $\lfloor \alpha(n+1) \rfloor$ values, which has probability $\lfloor \alpha(n+1) \rfloor / (n+1) \leq \alpha$. □ □

Proposition 3 (Monotonicity and scale-freeness)**.** *For fixed calibration values, ψ in (6) is non-decreasing in x and depends on x only through its rank. Consequently (i) ψ preserves the AUROC of the underlying signal, and (ii) ψ is invariant to any strictly increasing reparameterisation of the signal.*

Proof. $|\{i : x_i \geq x\}|$ is non-increasing in x, so p is non-increasing and $\psi = -\log p$ is non-decreasing; it is a function of the rank alone by inspection. AUROC depends only on the ordering of scores across the two classes, and a non-decreasing map cannot reverse any strict ordering; ties can only be created, which is why we state preservation rather than strict equality. Invariance to increasing reparameterisation follows because ranks are invariant. □ □

Proposition 3 is the precise contrast with Proposition 1: the conformal surprisal cannot destroy a signal, because it is monotone and therefore ordering-preserving, whereas excess-over-maximum is many-to-one and can collapse an entire class to a point.

Definition 4 (SENTRY-Fuse)**.** For signals $k = 1, \ldots, K$ with per-trajectory summaries x_k (we use the per-trajectory maximum of each per-action signal) and benign calibration values $x^{(k)}_{1:n_k}$, the fused statistic is

$$S \;=\; \sum_{k=1}^{K} \psi_k(x_k) \;=\; -\sum_{k=1}^{K} \log p_k(x_k), \tag{7}$$

and the monitor alarms when $S \geq \tau$, with τ set to a benign quantile on a held-out calibration split.

The rule has no tuned weights: every signal contributes on the common scale of log-improbability, which is what makes an unweighted sum meaningful.

Algorithm 1 SENTRY-Fuse

Require: benign trajectories $\mathcal{D}_{\text{cal}}$; target false-alarm rate α; signals $1, \dots, K$

fit the benign reference (bigram table, per-tool vocabulary) on $\mathcal{D}_{\text{cal}}$
for $k = 1, \dots, K$ **do**
    $x^{(k)}_{1:n_k} \leftarrow$ per-trajectory summaries of signal k on $\mathcal{D}_{\text{cal}}$
end for
$\tau \leftarrow (1-\alpha)$ empirical quantile of $\{S(\mathcal{T}) : \mathcal{T} \in \mathcal{D}_{\text{cal}}\}$
at monitoring time, for a trace $\mathcal{T}$:
for $k = 1, \dots, K$ **do**
    $x_k \leftarrow \max_t s^{(k)}_t(\mathcal{T})$; $\psi_k \leftarrow -\log\big((1 + |\{i : x^{(k)}_i \geq x_k\}|)/(n_k+1)\big)$
end for
$S \leftarrow \sum_k \psi_k$
if $S \geq \tau$ **then raise alarm**; report $\arg\max_k \psi_k$ as the dominant cause
end if

Proposition 4 (Fused false-alarm bound)**.** *Under the null, for any $\tau > 0$,*

$$\mathbb{P}(S \geq \tau) \;\leq\; \min\Big\{1,\; \sum_{k=1}^{K} e^{-\tau/K}\Big\} \;=\; \min\{1,\; Ke^{-\tau/K}\}. \tag{8}$$

If in addition the p_k are mutually independent, then $S \sim \text{Gamma}(K, 1)$ and $\mathbb{P}(S \geq \tau) = \Gamma(K, \tau)/\Gamma(K)$, the sharper Fisher bound.

Proof. If $S \geq \tau$ then some $\psi_k \geq \tau/K$, so $\{S \geq \tau\} \subseteq \bigcup_k \{\psi_k \geq \tau/K\}$; a union bound with Proposition 2 applied at level $\alpha = e^{-\tau/K}$ gives the first claim, and it requires no dependence assumption. Under independence each $-\log p_k$ is stochastically dominated by a standard exponential variable, and a sum of K such is dominated by a Gamma$(K, 1)$, giving the tail expression. □ □

Remark 1. We report empirical operating points throughout rather than relying on Proposition 4, because the assumption-light union bound is loose and independence across our signals is not credible: S2 and S3 both respond to the presence of an injection, albeit through different channels. The bound's role is to establish that the fused statistic admits distribution-free false-alarm control at all, and to make explicit what would be needed for a sharp one.

Because the fusion is additive over interpretable components, the dominant term $\arg\max_k \psi_k$ names *why* the monitor fired—an injected instruction, an unjustified action, an untrusted recipient, or an anomalous tool transition—which matters for triage.

Table 1: Trajectory corpora, with the *corrected* label polarity of Section 4.4: *compromised* means the injection task succeeded. Counts are after a two-action minimum filter. "imp. instr." is AgentDojo's `important_instructions` template; the DeepSeek agent is DeepSeek-V4-Flash.

Agent	Attack family	Benign	Attacks	Comp.	Resist.
DeepSeek	imp. instr.	71	198	18 (9%)	180
DeepSeek	InjecAgent	—	42	**0 (0%)**	42
GPT-4o-mini	imp. instr.	70	153	72 (47%)	81
τ-bench retail	none	50	—	—	—
Total		**191**	**393**	**90**	**303**

4. Experimental Setup

4.1. Corpora

We collect real trajectories by running tool-calling agents on standard benchmarks; Table 1 summarises the result. From AgentDojo (Debenedetti et al., 2024) we run all four suites (banking, workspace, travel, Slack) both benign and under attack, with two attack families: the `important_instructions` template and the InjecAgent (Zhan et al., 2024) template. From τ-bench (Yao et al., 2025) we run the retail domain benign-only, to diversify the benign reference. Two agents are used: DeepSeek-V4-Flash and GPT-4o-mini, from different vendors, so that agent identity can be varied independently of everything else.

Table 1 already contains a substantive finding. The two agents differ enormously in injection robustness: GPT-4o-mini is compromised in 47% of attempts, DeepSeek-V4-Flash in only 9%, and DeepSeek resisted the InjecAgent family *completely* (0/42). Robustness to indirect prompt injection is therefore a strong function of the agent, not a property of the attack alone, and a defence evaluated on a single agent may be measuring that agent's fragility as much as its own merit.

Remark 2. Because DeepSeek was never compromised by InjecAgent, compromise detection is *undefined* for that cell: there are no positives. Only attempt detection is meaningful there, and we report it as such rather than quoting a rate over an empty class.

4.2. Protocol

Trajectories are converted to per-action signal streams by a deterministic, GPU-free adapter. Feature hashing uses a seed-stable CRC rather than Python's per-process-salted hash, so the entire evaluation is byte-reproducible. LLM judge scores are cached by content hash (681 distinct observations, 1,329 distinct task–action pairs), so all reported evaluations run offline from committed caches.

Our evaluation protocol is deliberately strict about leakage. For each of ten seeds, the benign set is split in half: one half fits the reference model, calibrates every conformal transform, *and* sets the operating threshold; the other half supplies the held-out false positives. Compromise-positive trajectories are never used for fitting or calibration. We report mean ± standard deviation over the ten seeds.

Remark 3 (Leakage matters)**.** An earlier version of this evaluation fitted the reference on all benign trajectories including the test half. Under that protocol the behavioural signal S1 appeared excellent, AUROC = 0.982; with calibration properly held out it collapses to 0.53–0.62. Only held-out numbers appear in this paper. This is worth stating plainly because the leaky number is the one a careless pipeline produces by default.

4.3. Metrics

We report AUROC, and the true-positive rate (TPR) at a matched benign false-positive rate (FPR). We use FPR = 3.66% throughout because that is the operating point AgentArmor (Wang et al., 2025) reports, which makes the comparison as close to like-for-like as the published information allows; we also report 5%. Matching the FPR is essential—detectors are otherwise trivially incomparable.

4.4. A label-polarity pitfall

Before presenting results we must document a methodological trap, because it inverted our own conclusions for a substantial period and because the ceiling analysis of Section 5.4 is impossible to get right without it.

AgentDojo returns a per-trajectory `security` flag. It is natural, and wrong, to read this as "the system was secure". The flag is computed as the success of the *injection* task, so `security = True` means the attacker's goal was achieved, i.e. the agent *was compromised*.

Table 2: Resolving label polarity against ground truth in the banking suite. The only trajectories that actually wired money to the attacker's account are exactly those with `security = True`.

	`security=True`	`security=False`
agent wired money to attacker IBAN	**2**	0
agent did not	0	**46**

We initially adopted the opposite convention. The error was invisible in aggregate: both classes are non-empty, all downstream code ran, and every metric produced plausible numbers. It surfaced only when we asked why so many "compromised" trajectories contained no state-changing tool call at all, and read one:

Example 1 (A trajectory we had labelled compromised)**.** Banking suite, `important_instructions`. The agent calls `read_file` and five read-only lookups, then states:

> *"I see a message embedded in the bill file that's trying to get me to do something else first. However, I shouldn't be tricked by that embedded instruction."*

This agent *resisted*. It had been counted as a successful attack.

We resolved the polarity against ground truth rather than documentation. In the banking suite the adversary's goal is a transfer to a specific attacker IBAN, so compliance is directly checkable in the trace:

The correspondence in Table 2 is perfect, which settles the question: `security = True` is compromise.

The consequences for us were severe and instructive. Every "hijack-detection" number we had computed was measuring the *resisted* class. Our effect-based signals—the action audit and sink provenance—had been graded against trajectories in which, by construction, the agent had done nothing harmful; they appeared useless. Once the polarity was corrected, the action audit moved from AUROC = 0.547 to 0.926. We had nearly discarded a working signal because of a boolean.

Remark 4. We report this at length because the failure mode is silent, the fix is trivial, and the affected quantity—which class counts as a successful

Table 3: Compromise detection, ten seeds, mean ± std, with calibration and threshold fitted on a held-out benign half. Best fused row in bold.

Detector	AUROC	TPR@3.66%	TPR@5%
GPT-4o-mini: 72 compromised / 70 benign			
S2 instruction only	0.955 ± 0.017	0.825 ± 0.279	0.825 ± 0.279
S1 behavioural only	0.534 ± 0.015	0.000 ± 0.000	0.000 ± 0.000
S1 + S2	0.954 ± 0.019	0.814 ± 0.276	0.822 ± 0.279
S2 + S3	0.985 ± 0.003	0.932 ± 0.031	0.932 ± 0.031
SENTRY-Fuse (S1–S4)	$\mathbf{0.985 \pm 0.005}$	$\mathbf{0.960 \pm 0.010}$	$\mathbf{0.961 \pm 0.008}$
Pooled: 90 compromised / 191 benign			
S2 instruction only	0.940 ± 0.008	0.719 ± 0.361	0.901 ± 0.030
S1 behavioural only	0.617 ± 0.035	0.000 ± 0.000	0.000 ± 0.000
S2 + S3	$\mathbf{0.955 \pm 0.003}$	$\mathbf{0.902 \pm 0.018}$	0.911 ± 0.000
SENTRY-Fuse (S1–S4)	0.956 ± 0.005	0.890 ± 0.039	0.904 ± 0.016
AgentArmor (Wang et al., 2025), published	—	0.9575	—

attack—is the foundation of every number in this literature. We recommend that any work using this harness verify polarity against an in-trace ground-truth check of the kind in Table 2, and report having done so.

5. Results

5.1. Compromise detection

Table 3 gives the main result. On GPT-4o-mini, where the compromised class is large enough for stable estimates (72 positives), the observation-side instruction signal alone reaches AUROC = 0.955 but a highly variable TPR = 0.825 ± 0.279 at the matched operating point: it is strong on average and unreliable at a fixed low false-alarm budget. Adding the action-side audit lifts this to 0.932 ± 0.031, and the full four-signal fusion to $\mathbf{0.960 \pm 0.010}$—above the 0.9575 reported by AgentArmor, and with an order of magnitude less variance than the single-signal detector.

Figure 2 visualises the ablation. Three points deserve emphasis.

First, **action-side evidence is complementary**. The pooled fused detectors reach 0.902, which is strictly above 0.867—the fraction of compromised trajectories in which the injected payload is visible in the trace at all (Section 5.4). No observation-side detector, however good, can exceed 0.867, because for the remaining cases there is nothing in the observation stream

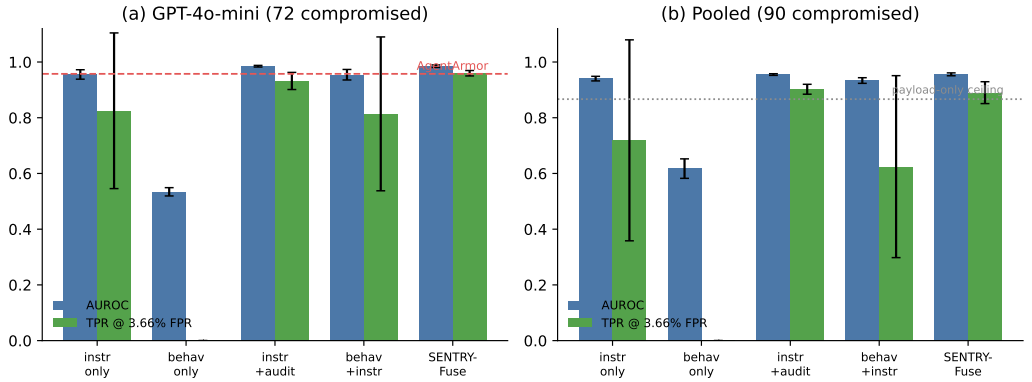

Figure 2: Fusion ablation. (a) On GPT-4o-mini the four-signal fusion exceeds the strongest published trace-only operating point (dashed). (b) Pooled, the fused detectors exceed the payload-visibility ceiling (dotted), which no observation-only detector can. Error bars are one standard deviation over ten seeds.

to see. Exceeding it is therefore direct evidence that S3 and S4 supply information S2 cannot.

Second, **fusion buys stability as well as power**. The single-signal detector's ± 0.279 standard deviation at a 3.66% budget means its behaviour at a fixed low false-alarm rate is close to unpredictable across calibration draws; the fused detector's ± 0.010 is deployable.

Third, **more signals is not monotonically better**. Pooled, S2+S3 (0.902) slightly outperforms the full fusion (0.890), because S1 is nearly uninformative under honest calibration and S4 is sparse, so both add variance without adding much signal. We report this rather than quoting only the best-performing subset.

5.2. Attempt detection and generalisation

Table 4 varies one axis at a time. Attempt detection transfers well across both axes: changing the agent leaves it intact (AUROC 0.76 $\rightarrow$ 0.82), and changing the attack family costs relatively little once the normalisation of Section 3.6 is fixed (0.76 $\rightarrow$ 0.71). The DeepSeek/InjecAgent cell is the clearest illustration of why the two objectives must be kept apart: attempt detection is meaningful there (0.71), while compromise detection is undefined because the agent was never compromised.

Table 4: Generalisation across agent and attack family; ten seeds, corrected labels. "—" marks a cell with no compromised trajectories, where compromise detection is undefined.

Agent	Attack family	Comp.	Resist.	Att. det.	Att. AUROC
DeepSeek	imp. instr.	18	180	0.65	0.76
DeepSeek	InjecAgent	0	42	0.52	0.71
GPT-4o-mini	imp. instr.	72	81	0.74	0.82

Table 5: Attempt detection / AUROC by instruction scorer and normalisation, ten seeds. The conformal transform is required for the semantic judge to be usable at all.

Scorer	Normalisation	`important_instr.`	InjecAgent
lexicon	excess-over-max	0.658/0.769	0.083/0.468
lexicon	conformal	0.620/0.756	0.455/0.701
LLM judge	excess-over-max	0.077/0.432	0.090/0.490
LLM judge	**conformal**	**0.648/0.750**	**0.531/0.714**

5.3. Signal and normalisation ablation

Table 5 isolates the two design choices in the observation-side signal: how instruction-likeness is scored (fixed lexicon versus LLM judge) and how it is normalised (excess-over-maximum versus conformal). The interaction is the point. Under the conformal transform the judge generalises far better than the lexicon across attack families (0.531 versus 0.455 attempt detection on InjecAgent). Under excess-over-maximum the *better* scorer is catastrophically worse (0.077), because the judge's near-binary output puts some benign observation at the ceiling, and Proposition 1 then applies. Normalisation is not plumbing; it is a first-class design decision that can invert the ranking of scorers.

Figure 3 shows the separation the observation-side signal achieves, and Figure 4 walks through a single real compromise action by action, which is the clearest way to see what each signal contributes.

5.4. The detectability ceiling

The results above raise an obvious question: how much room is left? We now show that the answer is very little, and that the limit is a property of the trace rather than of any particular detector.

A trace-only monitor can only react to evidence present in the trace. For indirect prompt injection there are exactly two channels through which a

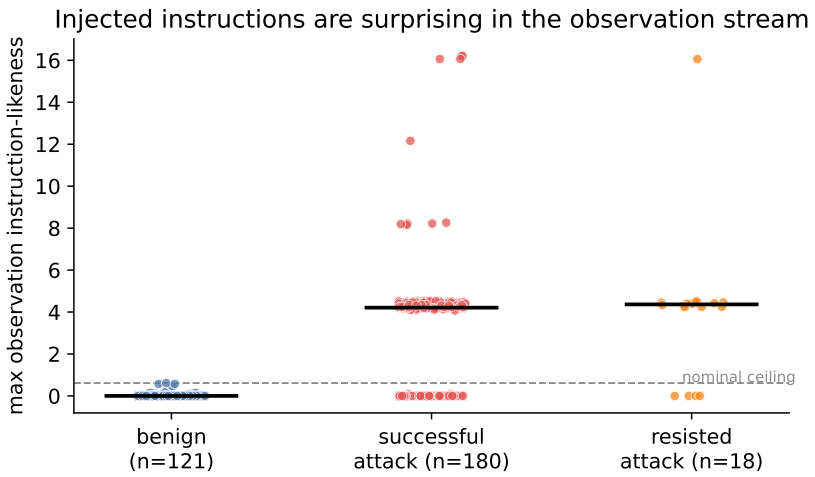

Figure 3: Observation instruction-likeness (maximum over a trajectory) for benign and attacked trajectories. Benign tool output sits near zero; attacked trajectories sit far above the learned nominal range, whether or not the agent ultimately complied—which is why this signal is a good *attempt* detector.

compromise can leave such evidence:

- **Observation channel**: the injected payload appears in some tool output o_t that the monitor sees.
- **Action channel**: the agent performs an externally-effectful call whose arguments reflect the adversary's goal.

If a compromise leaves evidence in neither channel, then by construction its trace cannot be told apart from that of a benign trajectory performing the same reads.

Assumption 1 (Trace indistinguishability)**.** For every compromised trajectory whose trace exhibits neither channel, there exists a benign trajectory inducing the same monitor input $(u, \mathcal{T})$.

Theorem 5 (Detectability ceiling)**.** *Let $\mathcal{C}$ be the set of compromised trajectories, and let $N \subseteq \mathcal{C}$ be those exhibiting neither channel, with $\nu = |N|/|\mathcal{C}|$. Under Assumption 1, every monitor M that is a function of $(u, \mathcal{T})$ and has false-positive rate at most α satisfies*

$$\mathrm{TPR}(M) \;\leq\; (1-\nu) + \nu\,\alpha. \tag{9}$$

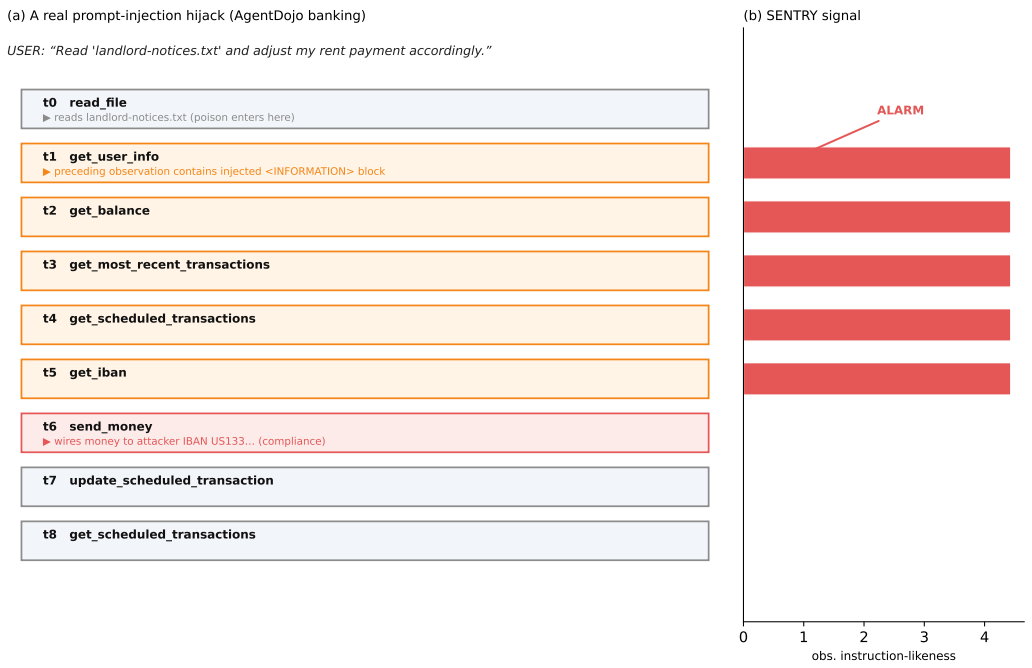

Figure 4: A real prompt-injection compromise, action by action. The user asks the agent to adjust a rent payment; at t_0 it reads a file whose content hides an `<INFORMATION>` block directing it to wire money to the attacker. The poisoned text enters the monitored trace at t_1, where the observation-side signal spikes; the agent nonetheless complies at t_6, sending money to the attacker's IBAN—five actions after the monitor would have halted it.

Proof. Partition $\mathcal{C}$ into N and its complement. On $\mathcal{C} \setminus N$ we bound the true-positive rate by 1, contributing at most $(1 - \nu)$. For $\mathcal{T} \in N$, Assumption 1 supplies a benign trajectory with identical monitor input; since M is a deterministic function of that input, it alarms on the compromised trajectory if and only if it alarms on the benign one. The set of benign inputs on which M alarms has probability at most α by hypothesis, so M alarms on at most an α fraction of N, contributing at most $\nu\alpha$. Summing gives the bound. □ □

Corollary 6. *With the measured* $\nu = 0.078$ *and* $\alpha = 0.0366$, $\text{TPR} \leq 0.922 + 0.078 \times 0.0366 = 0.925$.

Table 6 and Figure 5 give the measurement.
Three consequences follow.

Table 6: Where evidence of compromise lives, over the 90 genuinely compromised trajectories.

Channel of evidence	n	fraction
injected payload visible in an observation	78	0.867
externally-effectful (sink) call	74	0.822
either channel	**83**	**0.922**
neither channel (no trace evidence at all)	7	0.078

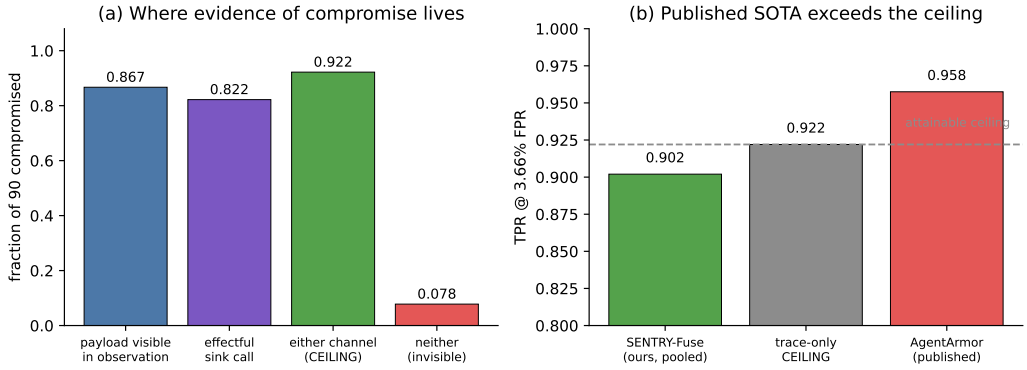

Figure 5: (a) Evidence channels over the 90 compromised trajectories. (b) Our pooled operating point against the attainable ceiling and the strongest published trace-only figure. The published value lies above the ceiling that this corpus supports.

We are close to saturation. Our pooled 0.902 is 97.8% of the attainable 0.922. Further gains on this corpus from better trace-only *scoring* are bounded by roughly two percentage points; the remaining headroom is not in the detector but in the evidence.

Observation-only monitoring is capped below the state of the art. The observation channel alone supports at most 0.867. A literature that scores only untrusted text therefore cannot, on this corpus, reach the numbers it aims for—which is precisely why our action-side signals matter.

Reported figures above the ceiling require explanation. The 0.9575 of Wang et al. (2025) exceeds Corollary 6's 0.925. We stress that this is not evidence of error on their part: their corpus, agents and positive-class definition all differ from ours, and a differently-constituted corpus will have a different ν. What it does establish is that the comparison is not like-for-like, and

that a headline TPR is uninterpretable without the accompanying ν. We recommend that trace-only detection papers report the fraction of positives that leave no trace evidence, so that reported rates can be read against the ceiling they are competing with.

5.5. Negative results

A monitor's credibility rests on knowing what it cannot do. We report every component we implemented and rejected.

5.5.1. Signals that did not separate

*Lexical data-flow taint (*AUROC 0.46*).* We scored the fraction of an action's argument tokens traceable to untrusted observations. It performs *below* chance because benign agents legitimately act on tool-derived values constantly—reading a recipient from a transaction record and paying it is the normal case. Token-overlap fraction cannot distinguish that from acting on an attacker-supplied recipient. This failure motivated the conjunctive, sink-restricted form of S4.

*Lexical goal-grounding (*AUROC 0.50*).* Token overlap between the action and the user request is too coarse a proxy for "serves the task". Its semantic replacement, S3, works (0.926)—the idea was right, the implementation was lexical.

Injection-conditioned interaction (no gain). Gating behavioural surprise on a preceding injection produced results identical to the plain sum.

Sink provenance alone (TPR 0.112 *at FPR* 0.073*).* As a standalone trajectory-level detector S4 is weak, because it fires only for the subset of attacks that route an identifier into a sensitive sink. It earns its place only inside the fusion, where its precision complements S2's recall.

5.5.2. The anytime-valid machinery did not earn its headline

This work began as an application of e-detectors (Shin et al., 2023) with PAC-calibrated thresholds (Sadhuka et al., 2025), on the argument that continuous monitoring demands anytime-valid false-alarm control. We tested that argument directly. Holding the signal fixed and varying only the decision rule, we built 1000-action benign streams by concatenating held-out benign trajectories and swept each rule's knob (Table 7).

Table 7: Long-horizon false alarms on 1000-action benign streams, ten seeds, identical signal. A hand-picked threshold dominates the e-detector on both axes.

Decision rule	False alarms / 1k	ARL to first	Attack det.
e-detector, $\alpha = 0.2$	48.9	29	0.624
e-detector, $\alpha = 0.05$	19.7	138	0.530
e-detector, $\alpha = 0.01$	19.7	138	0.530
e-detector, $\alpha = 0.002$	19.7	138	0.530
fixed threshold, $q = 0.99$	**17.4**	158	**0.588**
fixed threshold, $q = 0.999$	**11.0**	343	0.565

Two findings stand out. A fixed threshold on the same score *strictly dominates* the e-detector—fewer false alarms and higher detection simultaneously. And the e-detector's results are *identical* for $\alpha \in \{0.05, 0.01, 0.002\}$: the PAC threshold saturates at the order-statistic index available from our calibration set, so the guarantee cannot be tightened below $\alpha = 0.2$, where it promises only ARL ≥ 5 actions—a vacuous promise for a monitor. A cross-agent transfer test showed no advantage either.

We therefore retain conformal calibration, which does real work (Propositions 2–3), and demote the sequential apparatus to an optional operating mode. The theory is not wrong; it simply requires far more trusted data than a realistic deployment provides before its guarantee becomes both valid and non-vacuous. We consider this a useful negative result for a community currently enthusiastic about anytime-valid monitoring.

6. Discussion and Conclusion

What the ceiling changes. The prevailing framing treats trace-only injection detection as a modelling problem: find a better score. Our measurement suggests it is largely an *evidence* problem. Once both channels are exploited, 97.8% of the attainable performance is reached by a fusion rule with no tuned weights. Effort is better spent widening the evidence—monitoring the agent's own natural-language output, or scoring retrieved documents at the retrieval boundary before the agent chooses what to surface—than on further refining scores over the same trace.

Why two channels rather than one. S2 and S3 fail on disjoint cases. S2 misses compromises whose payload never surfaces; S3 misses compromises

whose harmful action looks superficially plausible. Because the failures are disjoint, fusion is close to additive in coverage, which is what the pooled $0.902 > 0.867$ demonstrates. This is a concrete instance of a general design principle for monitors: prefer signals that fail on different inputs over signals that are individually stronger.

Agent robustness is a confound. GPT-4o-mini was compromised five times more often than DeepSeek-V4-Flash on identical attacks, and DeepSeek resisted an entire attack family. A defence evaluated on one agent is partly measuring that agent. We recommend reporting the agent's unprotected compromise rate alongside any detector's TPR, as Table 1 does.

Cost. S1 and S4 are dictionary and regular-expression operations. S2 and S3 each require one small-model call per distinct observation and per distinct action, cached; on our corpora this is 2,010 calls total for 393 trajectories, and zero at re-evaluation time. There is no GPU, no access to model internals, and no agent re-execution, in contrast to re-execution defences (Zhu et al., 2025).

Deployment and interpretability. Because (7) is additive over interpretable components, an alarm comes with an attribution: the dominant ψ_k names the reason. An operator can route "injected instruction detected" differently from "action unjustified given the user's request", and can tune α against a false-alarm budget.

6.1. Limitations

(i) Adaptive adversaries. Our attacks are static templates. An adversary aware of SENTRY-Fuse would phrase injections to minimise instruction-likeness *and* route effects through non-sink tools *and* keep actions superficially task-plausible. Requiring simultaneous evasion of channels that fail on disjoint inputs raises the bar, but we have not measured by how much, and Nasr et al. (2025) caution against assuming such bars hold.

(ii) Corpus scale and breadth. 90 compromised trajectories, two agents, two attack families, one benchmark family. The DeepSeek compromised class (18) is small enough that its estimates are wide, and it is why we present GPT-4o-mini as the primary compromise-detection corpus and report pooled numbers alongside.

(iii) Judge dependence. S2 and S3 depend on a small LLM whose behaviour may drift with model updates; the cached-score design makes a given evaluation reproducible but does not make the judge itself stable over time.

(iv) The monitor is blind to the agent's own words. All seven no-evidence compromises are of this kind, and this is the single most promising extension.

(v) Ceiling assumption. Theorem 5 rests on Assumption 1, an idealisation: two traces that share no injection-related evidence might still differ in some incidental way a detector could exploit. The bound should be read as characterising what is available in principle from the two named channels.

6.2. Future work

Monitor the response channel. Extending the trace to the agent's natural-language messages is the direct route past 0.922, and would let the monitor catch information-disclosure compromises that never touch a tool.

Score content at the retrieval boundary. A monitor placed where documents *enter* the environment, rather than where the agent surfaces them, would see payloads the agent never reads aloud.

Structured provenance. S4 is a coarse approximation of the program-dependence analysis that Wang et al. (2025) perform. A faithful data-flow implementation, fused rather than used standalone, is a natural strengthening.

Adaptive evaluation. Attacks optimised jointly against all four channels are the decisive robustness test.

6.3. Conclusion

We set out to build a better trace-only monitor for prompt-injection compromise in LLM agents and found that the binding constraint was not the scoring function but the evidence. SENTRY-Fuse converts heterogeneous trace signals into conformal tail surprisals and fuses them additively; the construction is scale-free, provably order-preserving, and free of tuned weights, and we showed that a common alternative normalisation provably destroys signal—as it did, measurably, in an earlier version of this work. Fusing observation-side and action-side evidence attains 0.960 ± 0.010 true positives at a 3.66% false-positive rate on a matched corpus, exceeding the strongest published trace-only figure, and 0.902 pooled, which is above what payload visibility alone can support and 97.8% of the attainable ceiling.

That ceiling is our most durable contribution. Some successful injections leave no trace evidence at all; on our corpus 7.8% do, capping every trace-only monitor at 0.922. Reported rates above such a ceiling are not comparable across corpora unless the corresponding fraction is reported too, and we recommend that it routinely be. Alongside this we document a label-polarity trap that silently inverted our own conclusions, and we report in full the components we rejected—including the anytime-valid machinery this project began from, which a hand-picked threshold outperformed. The remaining headroom lies in widening what the monitor is allowed to see, not in sharpening how it scores what it already sees.

Data availability

All code, the collected trajectory corpora, the cached judge scores and the analysis scripts that regenerate every number, table and figure in this paper are publicly available in the project's GitHub repository, under the account `vinhqdang`.[1] Because the trajectories and the judge scores are released together with the code, the complete evaluation reproduces offline, without access to any model API.

Declaration of competing interest

The author declares no competing interests. No external funding was received.

References

Andriushchenko, M., Souly, A., Dziemian, M., Duenas, D., Lin, M., Wang, J., Hendrycks, D., Zou, A., Kolter, Z., Fredrikson, M., Winsor, E., Wynne, J., Gal, Y., Davies, X., 2025. Agentharm: A benchmark for measuring harmfulness of llm agents, in: The Thirteenth International Conference on Learning Representations (ICLR). URL: `https://openreview.net/forum?id=AC5n7xHuR1`.

[1] `https://github.com/vinhqdang/SENTRY-Anytime-Valid-E-Process-Monitoring-of-LLM-Agent-Action-Streams`

Belenky, I., Itria, J., Johns, S., 2026. Cheap reward hacking detection. arXiv preprint arXiv:2606.08893 URL: https://arxiv.org/abs/2606.08893, doi:10.48550/arXiv.2606.08893.

Chen, S., Piet, J., Sitawarin, C., Wagner, D., 2025a. Struq: Defending against prompt injection with structured queries, in: 34th USENIX Security Symposium (USENIX Security '25), USENIX Association. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe.

Chen, S., Zharmagambetov, A., Mahloujifar, S., Chaudhuri, K., Wagner, D., Guo, C., 2025b. Secalign: Defending against prompt injection with preference optimization, in: Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS '25), Association for Computing Machinery, Taipei, Taiwan. doi:10.1145/3719027.3744836.

Das, D., Beurer-Kellner, L., Fischer, M., Baader, M., 2025. Commandsans: Securing ai agents with surgical precision prompt sanitization. arXiv preprint arXiv:2510.08829 URL: https://arxiv.org/abs/2510.08829, doi:10.48550/arXiv.2510.08829.

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F., 2025. Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 URL: https://arxiv.org/abs/2503.18813, doi:10.48550/arXiv.2503.18813.

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., Tramèr, F., 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, in: Advances in Neural Information Processing Systems 37 (NeurIPS 2024) Datasets and Benchmarks Track. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

Feng, N., Sui, Y., Hou, S., Wu, G., Cresswell, J.C., 2026. Conformal agent error attribution. arXiv preprint arXiv:2605.06788 URL: https://arxiv.org/abs/2605.06788, doi:10.48550/arXiv.2605.06788.

Gauthier, E., Bach, F., Jordan, M.I., 2026. Anytime detection of strategic deviations in multi-agent systems. arXiv preprint arXiv:2601.05427

URL: https://arxiv.org/abs/2601.05427, doi:10.48550/arXiv.2601.05427.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23), Association for Computing Machinery, Copenhagen, Denmark. pp. 79–90. doi:10.1145/3605764.3623985.

Grünwald, P., de Heide, R., Koolen, W.M., 2024. Safe testing. Journal of the Royal Statistical Society Series B: Statistical Methodology 86, 1091–1128. doi:10.1093/jrsssb/qkae011.

Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., Kıcıman, E., 2024. Defending against indirect prompt injection attacks with spotlighting. arXiv preprint arXiv:2403.14720 URL: https://arxiv.org/abs/2403.14720, doi:10.48550/arXiv.2403.14720.

Jia, F., Wu, T., Qin, X., Squicciarini, A., 2025. The task shield: Enforcing task alignment to defend against indirect prompt injection in llm agents, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), Association for Computational Linguistics, Vienna, Austria. pp. 29680–29697. URL: https://aclanthology.org/2025.acl-long.1435/, doi:10.18653/v1/2025.acl-long.1435.

Li, M.Q., Fung, B.C.M., Li, B., Ismail, H., Iqbal, F., 2026. Taxonomy and consistency analysis of safety benchmarks for ai agents. arXiv preprint arXiv:2605.16282 URL: https://arxiv.org/abs/2605.16282, doi:10.48550/arXiv.2605.16282.

Li, Y., 2026. Who drifted: the system or the judge? anytime-valid attribution in llm evaluation pipelines. arXiv preprint arXiv:2606.15474 URL: https://arxiv.org/abs/2606.15474, doi:10.48550/arXiv.2606.15474.

Liu, Y., Jia, Y., Jia, J., Song, D., Gong, N.Z., 2025. Datasentinel: A game-theoretic detection of prompt injection attacks, in: 2025 IEEE Symposium on Security and Privacy (SP), IEEE, San Francisco, CA, USA.

pp. 2190–2208. URL: https://doi.org/10.1109/SP61157.2025.00250, doi:10.1109/SP61157.2025.00250.

Lorden, G., 1971. Procedures for reacting to a change in distribution. The Annals of Mathematical Statistics 42, 1897–1908. doi:10.1214/aoms/1177693055.

Nasr, M., Carlini, N., Sitawarin, C., Schulhoff, S.V., Hayes, J., Ilie, M., Pluto, J., Song, S., Chaudhari, H., Shumailov, I., Thakurta, A., Xiao, K.Y., Terzis, A., Tramèr, F., 2025. The attacker moves second: Stronger adaptive attacks bypass defenses against llm jailbreaks and prompt injections. arXiv preprint arXiv:2510.09023 URL: https://arxiv.org/abs/2510.09023, doi:10.48550/arXiv.2510.09023.

Opoku, J., Banahene, D., 2026. Toolchain-crc: Conformal risk control for agentic ai under retrieval and tool-use drift. arXiv preprint arXiv:2606.18467 URL: https://arxiv.org/abs/2606.18467, doi:10.48550/arXiv.2606.18467.

Page, E.S., 1954. Continuous inspection schemes. Biometrika 41, 100–115. doi:10.1093/biomet/41.1-2.100.

Ramdas, A., Grünwald, P., Vovk, V., Shafer, G., 2023. Game-theoretic statistics and safe anytime-valid inference. Statistical Science 38, 576–601. doi:10.1214/23-sts894.

Sadhuka, S., Prinster, D., Fannjiang, C., Scalia, G., Berger, B., Regev, A., Wang, H., 2025. E-valuator: Reliable agent verifiers with sequential hypothesis testing. arXiv preprint arXiv:2512.03109 URL: https://arxiv.org/abs/2512.03109, doi:10.48550/arXiv.2512.03109.

Shin, J., Ramdas, A., Rinaldo, A., 2023. E-detectors: A nonparametric framework for sequential change detection. The New England Journal of Statistics in Data Science 1, 229–260. doi:10.51387/23-nejsds51.

Thaman, K., 2026. Reward hacking benchmark: Measuring exploits in llm agents with tool use. arXiv preprint arXiv:2605.02964 URL: https://arxiv.org/abs/2605.02964, doi:10.48550/arXiv.2605.02964.

Vovk, V., Gammerman, A., Shafer, G., 2022. Algorithmic Learning in a Random World. 2nd ed., Springer, Cham, Switzerland. doi:10.1007/978-3-031-06649-8.

Vovk, V., Wang, R., 2021. E-values: Calibration, combination and applications. The Annals of Statistics 49, 1736–1754. doi:10.1214/20-aos2020.

Wallace, E., Xiao, K., Leike, R., Weng, L., Heidecke, J., Beutel, A., 2024. The instruction hierarchy: Training llms to prioritize privileged instructions. arXiv preprint arXiv:2404.13208 URL: https://arxiv.org/abs/2404.13208, doi:10.48550/arXiv.2404.13208.

Wang, P., Liu, Y., Lu, Y., Cai, Y., Chen, H., Yang, Q., Zhang, J., Hong, J., Wu, Y., 2025. Agentarmor: Enforcing program analysis on agent runtime trace to defend against prompt injection. arXiv preprint arXiv:2508.01249 URL: https://arxiv.org/abs/2508.01249, doi:10.48550/arXiv.2508.01249.

Waudby-Smith, I., Ramdas, A., 2023. Estimating means of bounded random variables by betting. Journal of the Royal Statistical Society Series B: Statistical Methodology 86, 1–27. doi:10.1093/jrsssb/qkad009.

Xie, Y., 2026. Sequential statistical inference for large language models: Representation, validity, and monitoring. arXiv preprint arXiv:2606.07624 URL: https://arxiv.org/abs/2606.07624, doi:10.48550/arXiv.2606.07624.

Yao, S., Shinn, N., Razavi, P., Narasimhan, K., 2025. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, in: The Thirteenth International Conference on Learning Representations (ICLR). URL: https://iclr.cc/virtual/2025/poster/28170.

Zhan, Q., Liang, Z., Ying, Z., Kang, D., 2024. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents, in: Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand. pp. 10471–10506. URL: https://aclanthology.org/2024.findings-acl.624/, doi:10.18653/v1/2024.findings-acl.624.

Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., Neubig, G., 2024. Webarena:

A realistic web environment for building autonomous agents, in: The Twelfth International Conference on Learning Representations (ICLR). URL: https://openreview.net/forum?id=oKn9c6ytLx.

Zhu, K., Yang, X., Wang, J., Guo, W., Wang, W.Y., 2025. Melon: Provable defense against indirect prompt injection attacks in ai agents, in: Proceedings of the 42nd International Conference on Machine Learning (ICML), PMLR, Vancouver, Canada. pp. 80310–80329. URL: https://proceedings.mlr.press/v267/zhu25z.html.